CS3081 Cloud Computing Mini Project: Report

Knowledge distillation and deployment of a command-parsing model on a local cloud

Kapil Kumar, Roll number ROLL_NUMBER

9 November 2026

Repository: https://github.com/Thegreatkk04/cloud-assignment (tag `final`)

How to read this report. Everything marked **Measured** was actually run by me and the command output is saved in `results/`. Everything marked **Not executed (design only)** was written and reviewed but could not be run before the deadline, mainly because the teacher labelling and the student training need several hours of CPU time and the cluster experiments depend on them. I have not put any number in a "not executed" section. Where I needed a value to show a calculation, it is marked **Assumption**.

0. Architecture overview

The system has four parts on one k3d node. **Storage**: SeaweedFS with its S3 gateway in namespace `data`, two buckets `datasets` and `models`. **Pipeline**: an indexed Kubernetes Job in namespace `pipeline` runs two labelling workers that call the teacher (phi3.5 on Ollama, running on the laptop, reached from the cluster through `host.k3d.internal`). **Serving**: a FastAPI service in namespace `app` behind the Traefik Ingress, which downloads the student model from the `models` bucket on start-up. **Automation**: a Makefile for the laptop and a GitHub Actions workflow for lint, tests, evaluation and secret scanning.

Data flow: `commands_pool.jsonl` -> two teacher shards -> `clean.py` -> `datasets/v1/train.jsonl` -> `train.py` (pythia-70m) -> `models/v1` in object storage and a GitHub Release -> service pods -> `POST /parse` through the Ingress.

1. Task 1: Object storage and parallel teacher labelling

Architecture. `cluster/k3d.yaml` creates a one-server cluster and maps port 8080 to the Traefik ingress and 8333 to a NodePort for the SeaweedFS S3 API, so laptop-side scripts and in-cluster pods both reach the same storage. `k8s/seaweedfs.yaml` runs one SeaweedFS pod (master, volume, filer, S3) in namespace `data`. `scripts/init_storage.py` creates the two buckets and uploads `commands_pool.jsonl`.

Implementation. `worker/label_worker.py` reads `SHARD_INDEX` and `SHARD_COUNT`, keeps the commands whose line number mod `SHARD_COUNT` equals `SHARD_INDEX`, sends each one to Ollama with `prompts/teacher_prompt.txt` (temperature 0, seed 42) and writes `{"id","text","teacher_raw"}` lines to `datasets/teacher/shard-N.jsonl`. The raw reply is stored untouched. Partial output is uploaded every 25 commands so a restarted worker does not lose everything. `k8s/worker-job.yaml` is an indexed Job with `completions: 2, parallelism: 2`; Kubernetes gives each pod its completion index, which the manifest maps to `SHARD_INDEX`, so the two workers run at the same time with indexes 0 and 1. `worker/clean.py` applies rules (a) to (e) in that order, counts each record under the first rule that rejects it, sorts by id, keeps the smallest id of each duplicate command, and writes `train.jsonl` plus its SHA-256.

Sharding explanation. Line number modulo shard count is a deterministic split: every id lands in exactly one shard, there are no gaps and no repeats, and the workers never have to talk to each other. Adding a worker only means changing `SHARD_COUNT` and `completions`. The downside is that the split is static; if one worker is slower the other cannot take over its remaining commands.

Results. *Measured:* `clean.py` is covered by three unit tests (`tests/test_clean.py`) that check every rejection reason, the duplicate rule, id ordering and that input = a + b + c + d + duplicates + kept. All pass (`results/pytest_verbose.txt`). *Not executed (design only):* the cluster, the SeaweedFS deployment, the two workers on the real 3,000 commands and therefore the counts table and the `train.sha256` checksum. I am not filling in the table with made-up values.

Limitations and improvements. Labelling 3,000 commands with a 3.8B model on a laptop CPU takes hours, which is the main reason this task was not completed in time. A work-queue (each worker pulls the next unlabelled id from a shared list) would balance stragglers better than static modulo sharding. Starting the teacher run on day one is the real fix.

2. Task 2: Training the student (sequence-level distillation)

Architecture. Each example is `Command: <command>\nJSON: <teacher json>`. The student (EleutherAI/pythia-70m) is trained with the normal causal-LM loss but with the prompt tokens masked (`-100`) so only the JSON part is learned. The prompt ends at `JSON:` with no trailing space, so the prompt tokens are an exact prefix of the full training string and the same prompt is used in `predict.py` and in the service.

Implementation. `training/config.yaml` holds the seed (42), learning rate (5e-5), batch size (8), epochs (3), max sequence length (128), the 1,500-record limit and the library versions. `make train` runs `training/train.py`, which reads only that file; `make train SUBSET=50` trains on the first 50 records. The loss of every epoch goes to `training/loss.csv`. The model, tokenizer, a `VERSION` file and `model.sha256` are saved to `models/v1/`, then archived as `v1.tar.gz` for the GitHub Release and uploaded to the `models` bucket with `scripts/upload_model.py`. `make train-v2` trains a second version with seed 7 into `models/v2/`.

Results. *Measured:* the untrained pythia-70m was run on the 100 validation commands with `training/predict.py` and scored with `tools/score.py` (`results/pred_base.jsonl`, `results/score_base.json`). It took 79.7 s, 0.80 s per command on 2 CPU threads.

model	json_validity	intent_acc	date	time	place	full_frame
untrained pythia-70m (measured)	0.0	0.0	0.0	0.0	0.0	0.0
teacher phi3.5	not executed					
student v1	not executed					

The base model outputs things like `- name: "calendar"` or a lone `{`, so 0 % validity is the honest starting point and it is what the CI quality gate is expected to fail on. *Not executed (design only):* training itself, so there is no `loss.csv`, no `models/v1`, no Release asset, no teacher score and no student score.

Explanation. (a) Sequence-level distillation trains the student on the teacher's generated text: the teacher's output tokens are the targets, and the loss is ordinary next-token cross-entropy on them. (b) Logit-based distillation trains the student to match the teacher's full probability distribution over the vocabulary at every position (soft targets, usually with a temperature and a KL loss). (c) It is

not possible here because the teacher runs behind Ollama, which returns only text, not logits, and phi3.5 and pythia-70m use different tokenizers and vocabularies, so even with logits the distributions would not line up token by token. (d) One real limitation of my approach: the student only ever sees the teacher's single greedy answer, so every teacher mistake is copied into the training data with no signal that it is uncertain, and 1,500 examples for a 70M model is small enough that slot rules like "copy the shortest phrase" will be learned only roughly.

Limitations and improvements. The pipeline is untested end to end. Next step is a 50-record smoke run, then the full run overnight. More epochs or a larger student would likely help `full_frame`.

3. Task 3: The inference service in a container

Architecture. `service/app.py` is a FastAPI app on port 8000 with `POST /parse`, `GET /healthz` and `GET /readyz`. At start-up a background thread downloads the model from `MODEL_URI` (an `s3://bucket/prefix`) through `S3_ENDPOINT` using the credentials from the Kubernetes Secret (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`), so `/readyz` returns 503 until the load finishes. `MODEL_VERSION` is echoed in `X-Model-Version`; `X-Served-By` is the pod name from the Downward API. `MAX_NEW_TOKENS` caps generation.

Implementation. Every model output goes through `model_runner.validate()`: must parse as JSON, have exactly the four keys, a known intent, and string-or-null slots. Anything else returns HTTP 500 with body `{"error": "invalid_model_output"}`; the service never returns broken JSON with a 200. With `FAULT_INJECT=1` the model output is replaced by the text `not json` before validation. `service/Dockerfile` starts from `python:3.11-slim`, installs the CPU-only torch wheel (`torch==2.4.1` from the PyTorch CPU index) and pinned versions of fastapi, uvicorn, pydantic, transformers, boto3 and numpy, and copies no weights.

Results. *Measured:* `tests/test_service_api.py` runs the app with a fake model runner and checks that `/healthz` is 200, `/readyz` is 503 before the model is set and 200 after, `/parse` returns exactly the four keys with both headers, and that `FAULT_INJECT=1` gives 500 with the exact error body. All three tests pass. *Not executed (design only):* the Docker build, so the image size is not measured, and the 50-request run at 1 CPU / 1 GiB, so no peak memory is reported.

Limitations and improvements. One uvicorn worker per pod; batching would increase throughput. The image size claim (under 2 GB because of the CPU-only wheel) is an expectation, not a measurement, until `docker build` is run.

4. Task 4: Kubernetes deployment, scaling and reliability

Architecture. In namespace `app`: a Deployment with 2 replicas, requests 250m CPU / 512Mi, limits 1 CPU / 1Gi, a liveness probe on `/healthz`, a readiness probe on `/readyz` with a high failure threshold so a slow model download does not kill the pod, and `RollingUpdate maxUnavailable=0, maxSurge=1` so one old pod always stays while a new one loads. A ClusterIP Service on port 80, a Traefik Ingress for `/parse`, and an HPA with min 1, max 2, target CPU 70 %.

Implementation. `make deploy` applies `k8s/deployment.yaml`, `service.yaml`, `ingress.yaml`, `hpa.yaml`. The rolling update is `kubectl -n app set env deploy/parser MODEL_VERSION=v2 MODEL_URI=s3://models/v2`, rollback is `kubectl rollout undo`. Recovery test is `kubectl delete pod` on one replica during the load test; cold start is `kubectl delete pods -l app=parser` three times with `kubectl get pods -w` timestamps and the "model v1 loaded from s3://models/v1" log line as evidence of the download.

Results. *Not executed (design only):* none of the eight cluster experiments were run, so there are no failure percentages, cold-start times or per-replica request counts.

Limitations and improvements. The cluster is one k3s node in Docker on one laptop; it shows scheduling and self-healing of pods but not fault tolerance across machines, and the HPA on a single

node only moves CPU between pods that share it. The readiness probe depends on object storage being up, so a storage outage makes every new pod stay NotReady; a cached copy of the model in a volume would soften that.

5. Task 5: Subnetting and network access control

Architecture and implementation. `scripts/subnet_calc.py` allocates the four subnets from 10.20.0.0/16, largest first, each starting right after the previous one, with the smallest prefix that fits (`usable = 2^(32-prefix) - 2`). `network/nacl.json` is the public-subnet NACL; the load balancer lives there. Working and tables are in `network/subnets.md`.

Results. *Measured* (`results/subnet_table.txt`, `results/nacl_check.txt`):

Subnet	Needed	Network	Prefix	Usable range	Broadcast	Usable
app	1000	10.20.0.0	/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1022
public	250	10.20.4.0	/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

NACL inbound: rule 10 deny all from 198.51.100.0/24; 100 allow TCP 80 from anywhere; 110 allow TCP 443 from anywhere; 120 allow TCP 22 from 10.20.5.128/27; 130 allow TCP 1024-65535 from 10.20.0.0/22 (replies from app). Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024-65535 to anywhere (replies to clients). `nacl_sim.py --check` on the six sample packets: **6 / 6 = 100 %**. Two more unit tests check edge cases (abusive range on port 80, SSH from app subnet denied, reply traffic from app allowed).

Explanation. A NACL is stateless: it keeps no connection table, so it cannot know that a packet coming back from the app server on the load balancer's ephemeral port belongs to a request it allowed a moment ago. Each direction therefore needs its own allow rule for the reply traffic (inbound 130 and outbound 110). Rule order matters because the lowest number is checked first and the first match wins. If the deny for 198.51.100.0/24 had a higher number than "allow 443 from anywhere", a packet from 198.51.100.9 to port 443 would match the allow first. Rule 10 sits below every allow rule, so packet S04 is denied by rule 10 while S01 from another address is allowed by rule 110.

Limitations and improvements. Rule 130 opens the whole ephemeral range from the app subnet, which is what a stateless ACL forces you to do; a stateful security group on the LB would be tighter. The hidden 20-packet set may include UDP or ICMP cases that the sample does not.

6. Task 6: Reproducibility, testing and CI/CD

Architecture and implementation. `make up` runs `scripts/up.sh` (cluster, namespaces, SeaweedFS, secrets from `.env`, buckets, image builds, image import, deploy); with `cp .env.example .env` and `make train` that is three commands from a fresh clone. `make down` deletes the cluster. `.github/workflows/ci.yml` has four jobs in order: `lint` (pyflakes), `test` (pytest), `evaluate` (downloads `v1.tar.gz` from the `model-v1` Release with `gh`, runs `predict.py` and then `score.py --min-validity 95` as the gate; a `workflow_dispatch` input `model_source=base` evaluates the untrained model instead so the gate fails) and `secrets` (gitleaks). `AI_USE.md` is in the repo root.

Results. *Measured:* `python -m pyflakes worker service training scripts tests` reports nothing (`results/lint.txt`); `python -m pytest -q tests` gives **13 passed** (`results/pytest.txt`, per-test list in `results/pytest_verbose.txt`). The measured 0 % validity of the base model (Task 2) is what the gate would reject. *Not executed:* the workflow has not run on GitHub yet, so there is no link to a failed gate run, and `make up` was not tried from a fresh clone.

Limitations and improvements. The evaluate job needs the Release asset to exist; until training is done the only runnable mode is `model_source=base`. Running `make up` on a clean machine will surely surface small path and timing problems that I could not catch here.

7. Task 7: Performance, cost and review

Load test. *Not executed:* no Locust run, so no p50/p95, throughput, error rate, `kubectl top` or cold-start numbers. The standard command is in the Makefile (`make load`).

Cost. `scripts/cost.py` implements the rate-card formula. Rates: 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour, each replica billed for its limits (1 CPU, 1 GiB). One replica per hour = 1 x 3.50 + 1 x 0.45 = 3.95 INR. Two replicas = 7.90 INR per hour. *Assumption, not measured:* a throughput of 1.0 request/s, i.e. 3,600 requests per hour. Cost per 1,000 requests = 7.90 / 3,600 x 1,000 = **2.19 INR** (`results/cost_assumed_rps1.txt`). Once a real throughput is measured, `make cost RPS=<value>` redoes the arithmetic. The unit test `tests/test_cost.py` checks the formula.

SLO. Target: 99 % of `/parse` requests return HTTP 200 within 30 s. Window: one standard load-test run (4 users, 2 minutes). Error budget = 1 % of the requests in that run; for example a run of 300 requests would allow 3 failures. *Not executed:* the budget used is unknown because no run was done.

Review table. The brief asks for a measured number in every evidence cell. Only the rows I actually measured have one; the rest say so.

Concern	Evidence	Limitation found	Improvement
Data and storage	clean.py rules covered by 3 passing tests; real counts not measured	teacher run not done, so no checksum	start labelling on day 1, use a work queue
Model quality	untrained base: 0.0 % validity, 0.0 % full_frame (100 examples)	student not trained	run `make train`, compare with teacher
Latency and throughput	base model: 0.80 s per command, single thread of 2 (not the service)	no Locust run	run `make load`, add batching
Resource utilisation	not measured	limits set (1 CPU / 1 GiB) but never observed	`kubectl top pods` during load
Reliability and recovery	not measured	single node, no multi-machine fault tolerance	multi-node k3d, PodDisruptionBudget
Security and cost	gitleaks config present; 2.19 INR per 1,000 req at an assumed 1 rps	cost based on an assumption	measure rps, right-size limits

8. Overall limitations

The honest summary is that the design, code, tests and network work are done and checked, while everything that needed the teacher model, hours of CPU training or the running cluster is not. I chose to leave those sections without numbers rather than fill them in, because the brief is clear that measured results and assumptions must be kept apart.